

Secure Items API — FastAPI + JWT Authentication

A RESTful microservice built with **FastAPI**, featuring user registration/login, **bcrypt** password hashing, **JWT Bearer** token authorization, **Pydantic** request validation, and full **CRUD** on a protected `items` resource. Interactive docs are auto-generated at `/docs`.

Project structure

```
.
├── main.py          # FastAPI app + all routes
├── auth.py          # password hashing + JWT create/verify + get_current_user
├── models.py        # SQLAlchemy ORM models (User, Item)
├── schemas.py       # Pydantic request/response schemas
├── crud.py          # DB access functions
├── database.py      # SQLAlchemy engine/session (SQLite)
├── requirements.txt
└── postman_collection.json
```

Setup

```
pip install -r requirements.txt
```

Run

```
uvicorn main:app --reload
```

- API base: <http://127.0.0.1:8000>
- Swagger UI: <http://127.0.0.1:8000/docs> ← take your screenshots here
- ReDoc: <http://127.0.0.1:8000/redoc>

Auth flow

1. `POST /auth/register` — create a user (`username` , `email` , `password`)
2. `POST /auth/login` — `form-data` `username` + `password` → returns `access_token`
3. Use the token as `Authorization: Bearer <token>` on all `/items` routes

(or click **Authorize** in Swagger UI and paste the token)

Endpoints

Method	Path	Auth required	Description
POST	/auth/register	No	Create a new user
POST	/auth/login	No	Get a JWT access token
GET	/auth/me	Yes	Current user profile
POST	/items	Yes	Create an item
GET	/items	Yes	List your items
GET	/items/{id}	Yes	Get one item
PUT	/items/{id}	Yes	Update an item
DELETE	/items/{id}	Yes	Delete an item

Postman

Import `postman_collection.json` into Postman. Run **Register** → **Login** (auto-saves the token to a collection variable) → then the `/items` requests.

Notes

- Passwords are hashed with `bcrypt (passlib)`, never stored in plain text.
- `JWT_SECRET_KEY` is read from an environment variable; a dev default is used if unset — **set a real secret before deploying**.
- `SQLite (app.db)` is used for simplicity; swap the `SQLALCHEMY_DATABASE_URL` in `database.py` for Postgres/MySQL in production.

Submission

1. Push this folder to a public GitHub repo.
2. Run the app locally, open `/docs`, and take 2–3 screenshots (Swagger UI overview, the Authorize dialog, a successful request).
3. Add the screenshots to the repo (e.g. in a `screenshots/` folder) and include

postman_collection.json .

4. Copy the repo's public URL and paste it into "Submit deliverable".